

UNIVERSIDADE ESTADUAL DO CENTRO OESTE

Ciência da Computação

Trabalho de Programação Orientada a Objetos

implementação de padrões GoF

Guilherme Oliveira Matos

10/11/2026

Service

Na primeira versão projeto feita na arquitetura MVC + DAO, os controller acumulavam varias responsabilidades como gerenciar a interface, as interações do usuário, validações de campo, regras de negócio e realizavam chamadas diretas ao DAO.

Então atravez do padrão Service foi feito a separação de responsabilidades, Agora os controladores passam a ter como função intermediar a interface e os serviços do sistema, toda a inteligência do negócio e as regras de domínio foram encapsuladas na camada de serviço.

Para exemplificar essa refatoração, a seguir o salvamento de um vendedor:

Versão antiga

```java
@FXML
public void salvarVendedor() {
    try {
        String cpf = txtCpf.getText();
        String nome = txtNome.getText();
        String email = txtEmail.getText();
        String senha = txtSenha.getText();

        // Validação manual
        if (cpf.trim().isEmpty() || nome.trim().isEmpty() || senha.trim().isEmpty()) {
            alerta("Erro de Validação", "CPF, Nome e Senha são campos obrigatórios.");
            return;
        }

        Vendedor vendedor = new Vendedor(cpf, nome, email, senha);

        // o controller consulta o DAO direto para decidir se Insere ou Altera
        if (vendedorDAO.selecionarPorCpf(cpf) == null) {
            if (vendedorDAO.inserir(vendedor)) {
                alerta("Sucesso", "Vendedor cadastrado com sucesso!");
            } else {
                alerta("Erro", "Não foi possível cadastrar o vendedor.");
            }
        } else {
            if (vendedorDAO.alterar(vendedor)) {
                alerta("Sucesso", "Vendedor atualizado com sucesso!");
            } else {
                alerta("Erro", "Não foi possível atualizar o vendedor.");
            }
        }

        limparCampos();
        atualizarTabela();
    } catch (Exception e) {
        alerta("Erro", "Ocorreu um erro ao salvar o vendedor: " + e.getMessage());
    }
}
```

Versão Nova

```java
@FXML
public void salvarVendedor() {
    try {
        String cpf = txtCpf.getText();
        String nome = txtNome.getText();
        String email = txtEmail.getText();
        String senha = txtSenha.getText();

        Vendedor vendedor = new Vendedor(cpf, nome, email, senha);

        // lógica de decisão inserir/alterar fica no Service
        vendedorService.salvarOuAtualizar(vendedor);[cite: 14]

        alerta("Sucesso", "Operação realizada com sucesso!");
        limparCampos();
        atualizarTabela();
    } catch (Exception e) {
        // trata qualquer erro pela Service
        alerta("Erro de Validação / Negócio", e.getMessage());[cite: 14]
    }
}
```

No versão antiga, o controlador realizava a checagem manual de String.trim().isEmpty() para CPF, nome e senha, consultava o VendedorDAO.selecionarPorCpf() para verificar a existência do cadastro e decidia condicionalmente se invocaria vendedorDAO.inserir() ou vendedorDAO.alterar(), Agora apenas coleta as informações, instancia o modelo `Vendedor` e passa a execução ao serviço invocando onde valida se os campos obrigatórios estão preenchidos, lançando `IllegalArgumentException` caso não, consulta a existência prévia do CPF no banco e executa a inserção ou alteração correspondente no `VendedorDAO`

Builder

Bem agora no padrão Builder ele foi escolhido por causa do Objeto carro ter muitos atributos, e o padrão ajuda com uma instanciação/criação do objeto de uma maneira mais legivel e “elegante”.

A principais mudança foi que antes, No método `salvarCarro()`, o objeto era instanciado de forma rígida pelo construtor cheio:

```java
Carro carro = new Carro(placa, modelo, marca, ano, preco, nomeArquivoImagem);
```

Agora A instanciação passou a utilizar a construção do Builder, tornando o código altamente legível e auto-explicativo:

```
Carro carro = new Carro.CarroBuilder()
        .comPlaca(placa)
        .comModelo(modelo)
        .comMarca(marca)
        .comAno(ano)
        .comPreco(preco)
        .comImagem(nomeArquivoImagem)
        .build();
```

Bem e para tudo isso funcionar foi feito a auteração no model com o Builder como uma classe estatica aninhada

```
public static class CarroBuilder {
        private String placa;
        private String modelo;
        private String marca;
        private int ano;
        private double preco;
        private String imagem;

public CarroBuilder comPlaca(String placa) {
...
```

Strategy

O padrão foi aplicado para pensando em corrigir uma “deficiencia” do projeto orginal, onde Vendas não possui valor de venda, os carros ate possuiam preços mas nada envolvendo as venda, então forão feitas algumas modificações sendo elas a adição de uma variavel Valor final nos Models, DAO e Banco, a modificação da interface para poder realmente realizar a venda que é feita atravez do padrão onde:

É selecionado a forma de pagamento na interface o controller le e processa com:

```
CalculoVendaStrategy estrategia;
String opcaoPagamento = cbTipoPagamento.getValue();

if (opcaoPagamento.contains("À Vista")) {
    estrategia = new DescontoAVistaStrategy();
} else if (opcaoPagamento.contains("Financiado")) {
    estrategia = new VendaFinanciadaStrategy();
} else {
    estrategia = new VendaNormalStrategy();
```

Os três metados são:

```
package br.unicentro.sistema_concessionaria.strategy;

public class VendaNormalStrategy implements CalculoVendaStrategy {
    @Override
    public double calcularValorFinal(double precoBase) {
        return precoBase;
    }
}
```

```
public class DescontoAVistaStrategy implements CalculoVendaStrategy {
    @Override
    public double calcularValorFinal(double precoBase) {
        return precoBase * 0.95; // 5% de desconto
    }
}
```

```
package br.unicentro.sistema_concessionaria.strategy;

public class VendaFinanciadaStrategy implements CalculoVendaStrategy {
    @Override
    public double calcularValorFinal(double precoBase) {
        return precoBase * 1.03; // 3% de acréscimo
    }
}
```

Assim formando o método de pagamento “modular” do projeto.

Facade

O ultimo padrão foi escolhido com a idea do literal significado do padrão que é fachafa, servindo como uma classe para simplificar o conportamento da venda, onde antes ela para fazer uma venda envolve múltiplos subsistemas (validar se o vendedor existe, verificar se o carro está disponível, calcular impostos/descontos, salvar a venda no banco e atualizar o status do carro para "Vendido"), Agora o facade unifica tudo em um único método:

VendaController antes:

```
@FXML
public void salvarVenda() {
    try {
        String placa = cbPlacaCarro.getValue();
        String comprador = txtNomeComprador.getText();
        LocalDate data = dpDataVenda.getValue();
        Vendedor logado = LoginController.getVendedorLogado();

        if (logado == null) {
            alerta("Erro", "Nenhum vendedor autenticado no sistema.");
            return;
        }

        CalculoVendaStrategy estrategia;
        String opcaoPagamento = (cbTipoPagamento != null && cbTipoPagamento.getValue() != null) ? cbTipoPagamento.getValue() : "Normal";

        if (opcaoPagamento.contains("À Vista")) {
            estrategia = new DescontoAVistaStrategy();
        } else if (opcaoPagamento.contains("Financiado")) {
            estrategia = new VendaFinanciadaStrategy();
        } else {
            estrategia = new VendaNormalStrategy();
        }

        if (vendaSelecionada == null) {
            Venda novaVenda = new Venda(0, placa, logado.getCpf(), comprador, data, 0.0);

            vendaService.registrarVendaComEstrategia(novaVenda, estrategia);[cite: 26]

            alerta("Sucesso", "Venda registrada com sucesso! Valor final: R$ " + String.format("%.2f", novaVenda.getValorFinal()));
        } else {
            vendaSelecionada.setPlaca(placa);
            vendaSelecionada.setCpf(logado.getCpf());
            vendaSelecionada.setNomeComprador(comprador);
            vendaSelecionada.setDataVenda(data);

            vendaService.registrarVendaComEstrategia(vendaSelecionada, estrategia);[cite: 26]
            alerta("Sucesso", "Venda atualizada com sucesso!");
        }

        limparCampos();
        atualizarTabela();
    } catch (Exception e) {
        alerta("Erro de Regra de Negócio", e.getMessage());
    }
}
```

VendasController agora:

```
private final ConcessionariaFacade concessionariaFacade = new ConcessionariaFacade();

@FXML
public void salvarVenda() {
    try {
        String placa = cbPlacaCarro.getValue();
        String comprador = txtNomeComprador.getText();
        LocalDate data = dpDataVenda.getValue();
        Vendedor logado = LoginController.getVendedorLogado();

        if (logado == null) {
            alerta("Erro", "Nenhum vendedor autenticado no sistema.");
            return;
        }

        String formaPagamento = (cbTipoPagamento != null && cbTipoPagamento.getValue() != null)
                ? cbTipoPagamento.getValue() : "Normal";

        if (vendaSelecionada == null) {
            Venda novaVenda = new Venda(0, placa, logado.getCpf(), comprador, data, 0.0);

            concessionariaFacade.registrarOuAtualizarVenda(novaVenda, formaPagamento);[cite: 27]

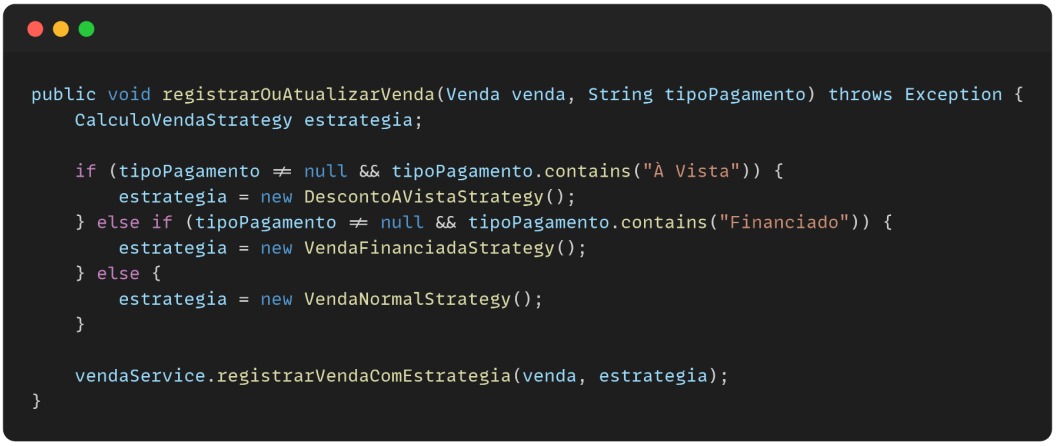
            alerta("Sucesso", "Venda registrada com sucesso! Valor final: R$ " + String.format("%.2f", novaVenda.getValorFinal()));
        } else {
            vendaSelecionada.setPlaca(placa);
            vendaSelecionada.setCpf(logado.getCpf());
            vendaSelecionada.setNomeComprador(comprador);
            vendaSelecionada.setDataVenda(data);

            concessionariaFacade.registrarOuAtualizarVenda(vendaSelecionada, formaPagamento);[cite: 27]

            alerta("Sucesso", "Venda atualizada com sucesso!");
        }

        limparCampos();
        atualizarTabela();
    } catch (Exception e) {
        alerta("Erro de Regra de Negócio", e.getMessage());
    }
}
```

Assim toda aquela lógica de decisão de qual strategy instanciar saiu de dentro do controlador da tela e foi para ConcessionariaFacade:



```
public void registrarOuAtualizarVenda(Venda venda, String tipoPagamento) throws Exception {
    CalculoVendaStrategy estrategia;

    if (tipoPagamento != null && tipoPagamento.contains("À Vista")) {
        estrategia = new DescontoAVistaStrategy();
    } else if (tipoPagamento != null && tipoPagamento.contains("Financiado")) {
        estrategia = new VendaFinanciadaStrategy();
    } else {
        estrategia = new VendaNormalStrategy();
    }

    vendaService.registrarVendaComEstrategia(venda, estrategia);
}
```